



```
print "Hello World"
```

Introduction to Python: Lambda Functions and Function Passing

Python is a versatile and expressive programming language that supports various programming paradigms. Two essential concepts in Python are lambda functions and the ability to pass functions as arguments to other functions. Understanding these concepts is vital for writing clean, concise, and efficient code, which can be particularly valuable in machine learning and data security.



by Mohd Iqbal Bhat



Understanding Lambda Functions

- Lambda functions, also known as anonymous functions, are small, nameless functions defined using the `lambda` keyword.
- They are used when you need a simple, one-liner function for a short period and don't want to define a regular function using `def`.

The syntax of a lambda function is:

```
lambda arguments: expression
```

Lambda functions can have multiple arguments but should contain only a single expression.

Examples of Lambda Functions

Example 1: Simple Lambda Function

```
add = lambda x, y: x + y
```

```
print(add(5, 3)) # Output: 8
```

In this example, we define a lambda function that takes two arguments `x` and `y` and returns their sum.



Using Lambda Functions with Built-in Functions

Lambda functions are often used in combination with built-in functions like `sorted()` to customize sorting.

Using Lambda Functions with `sorted()`

```
students = [  
    {"name": "Alice", "grade": 85},  
    {"name": "Bob", "grade": 92},  
    {"name": "Charlie", "grade": 78}  
]
```

```
sorted_students = sorted(students, key=lambda student: student["grade"])
```

```
print(sorted_students)
```

In this example, we use a lambda function as the `key` argument in `sorted()`, allowing us to sort a list of dictionaries based on a specific key.

Passing Functions as Arguments

Python treats functions as first-class objects, which means you can pass functions as arguments to other functions. This is a powerful concept and can lead to more flexible and reusable code.

Example: Using Functions as Arguments

```
def apply(func, num_list):  
    result = []  
    for num in num_list:  
        result.append(func(num))  
    return result
```

```
def square(x):  
    return x * x
```

```
def cube(x):  
    return x * x * x
```

```
numbers = [1, 2, 3, 4, 5]
```

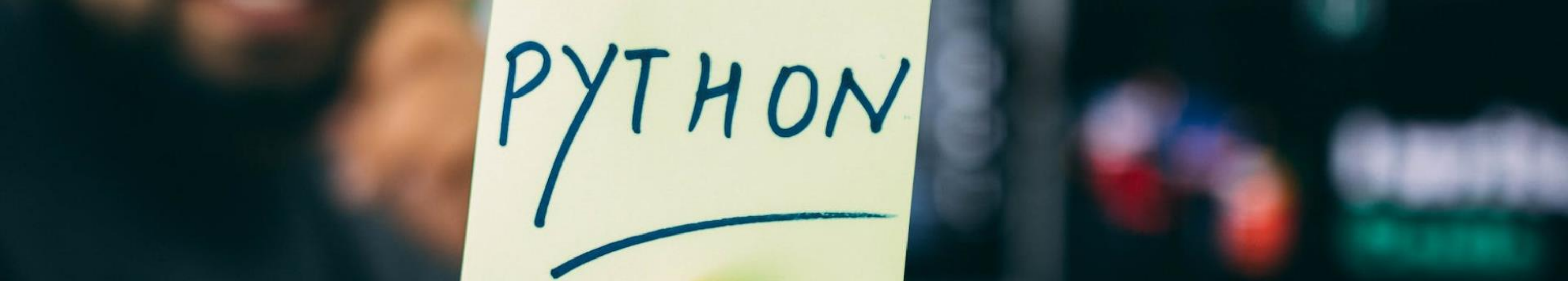
```
squared_numbers = apply(square, numbers)
```

```
cubed_numbers = apply(cube, numbers)
```

```
print(squared_numbers)
```

```
print(cubed_numbers)
```

In this example, we define an `apply` function that takes another function `func` as an argument and applies it to a list of numbers. This approach is useful for applying various operations to the same data without duplicating code.



PYTHON

Conclusion

Lambda functions and the ability to pass functions as arguments are essential Python concepts. They enhance code readability, reusability, and flexibility, making Python a powerful and expressive language. In fields such as machine learning and data security, these concepts can be applied to streamline data processing, algorithm customization, and more.

As an Assistant Professor, you can introduce these concepts to your students to help them become proficient Python programmers. These skills will be valuable in various domains, including your research interests in cryptography, blockchain, and machine learning.

I hope this lecture helps you and your students gain a deeper understanding of lambda functions and function passing in Python.